

本章将定制并实现更加基本，且更为常用的两类数据结构——栈与队列。与此前介绍的向量和列表一样，它们也属于线性序列结构，故其中存放的数据对象之间也具有线性次序。相对于一般的序列结构，栈与队列的数据操作范围仅限于逻辑上的特定某端。然而，得益于其简洁性与规范性，它们既成为构建更复杂、更高级数据结构的基础，同时也是算法设计的基本出发点，甚至常常作为标准配置的基本数据结构以硬件形式直接实现。因此无论就工程或理论而言，其基础性地位都是其它结构无法比拟的。

在信息处理领域，栈与队列的身影随处可见。许多程序语言本身就是建立于栈结构之上，无论PostScript或者Java，其实时运行环境都是基于栈结构的虚拟机。再如，网络浏览器多会将用户最近访问过的地址组织为一个栈。这样，用户每访问一个新页面，其地址就会被存放至栈顶；而用户每按下一次“后退”按钮，即可沿相反的次序返回此前刚访问过的页面。类似地，主流的文本编辑器也大都支持编辑操作的历史记录功能，用户的编辑操作被依次记录在一个栈中。一旦出现误操作，用户只需按下“撤销”按钮，即可取消最近一次操作并回到此前的编辑状态。

在需要公平且经济地对各种自然或社会资源做管理或分配的场合，无论是调度银行和医院的服务窗口，还是管理轮耕的田地和轮伐的森林，队列都可大显身手。甚至计算机及其网络自身内部的各种计算资源，无论是多进程共享的CPU时间，还是多用户共享的打印机，也都需要借助队列结构实现合理和优化的分配。

相对于向量和列表，栈与队列的外部接口更为简化和紧凑，故亦可视作向量与列表的特例，因此C++的继承与封装机制在此可以大显身手。得益于此，本章的重点将不再拘泥于对数据结构内部实现机制的展示，并转而更多地从其外部特性出发，结合若干典型的实际问题介绍栈与队列的具体应用。

在栈的应用方面，本章将在1.4节的基础上，结合函数调用栈的机制介绍一般函数调用的实现方式与过程，并将其推广至递归调用。然后以降低空间复杂度的目标为线索，介绍通过显式地维护栈结构解决应用问题的典型方法和基本技巧。此外，还将着重介绍如何利用栈结构，实现基于试探回溯策略的高效搜索算法。在队列的应用方面，本章将介绍如何实现基于轮值策略的通用循环分配器，并以银行窗口服务为例实现基本的调度算法。

§ 4.1 栈

4.1.1 ADT接口

■ 入栈与出栈

栈（stack）是存放数据对象的一种特殊容器，其中的数据元素按线性的逻辑次序排列，故也可定义首、末元素。不过，尽管栈结构也支持对象的插入和删除操作，但其操作的范围仅限于栈的某一特定端。也就是说，若约定新的元素只能从某一端插入其中，则反过来也只能从这一端删除已有的元素。禁止操作的另一端，称作盲端。



图4.1 一摞椅子即是一个栈

如图4.1所示，数把椅子叠成一摞即可视作一个栈。为维持这一放置形式，对该栈可行的操作只能在其顶部实施：新的椅子只能叠放到最顶端；反过来，只有最顶端的椅子才能被取走。因此比照这类实例，栈中可操作的一端更多地称作栈顶（**stack top**），而另一无法直接操作的盲端则更多地称作栈底（**stack bottom**）。

作为抽象数据类型，栈所支持的操作接口可归纳为表4.1。其中除了引用栈顶的**top()**等操作外，如图4.2所示，最常用的插入与删除操作分别称作入栈（**push**）和出栈（**pop**）。

■ 后进先出

由以上关于栈操作位置的约定和限制不难看出，栈中元素接受操作的次序必然始终遵循所谓“后进先出”（**last-in-first-out, LIFO**）的规律：从栈结构的整个生命期来看，更晚（早）出栈的元素，应为更早（晚）入栈者；反之，更晚（早）入栈者应更早（晚）出栈。

4.1.2 操作实例

表4.2给出了一个存放整数的栈从被创建开始，按以上接口实施一系列操作的过程。

表4.2 栈操作实例

操 作	输 出	栈（左侧为栈顶）
Stack()		
empty()	true	
push(5)		5
push(3)		3 5
pop()	3	5
push(7)		7 5
push(3)		3 7 5
top()	3	3 7 5
empty()	false	3 7 5

操 作	输 出	栈（左侧为栈顶）
push(11)		11 3 7 5
size()	4	11 3 7 5
push(6)		6 11 3 7 5
empty()	false	6 11 3 7 5
push(7)		7 6 11 3 7 5
pop()	7	6 11 3 7 5
pop()	6	11 3 7 5
top()	11	11 3 7 5
size()	4	11 3 7 5

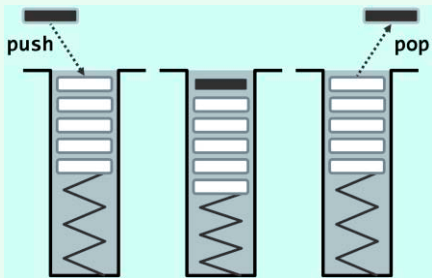


图4.2 栈操作

表4.1 栈ADT支持的操作接口

操作接口	功 能
size()	报告栈的规模
empty()	判断栈是否为空
push(e)	将e插至栈顶
pop()	删除栈顶对象
top()	引用栈顶对象

4.1.3 Stack模板类

既然栈可视为序列的特例，故只要将栈作为向量的派生类，即可利用C++的继承机制，基于2.2.3节定义的向量模板类实现栈结构。当然，这里需要按照栈的习惯，对各接口重新命名。

按照表4.1所列的ADT接口，可描述并实现Stack模板类如代码4.1所示。

```
1 #include "../Vector/Vector.h" //以向量为基类，派生出栈模板类
2 template <typename T> class Stack: public Vector<T> { //将向量的首/末端作为栈底/顶
3 public: //size()、empty()以及其它开放接口，均可直接沿用
4     void push(T const& e) { insert(size(), e); } //入栈：等效于将新元素作为向量的末元素插入
5     T pop() { return remove(size() - 1); } //出栈：等效于删除向量的末元素
6     T& top() { return (*this)[size() - 1]; } //取顶：直接返回向量的末元素
7 };
```

代码4.1 Stack模板类

既然栈操作都限制于向量的末端，参与操作的元素没有任何后继，故由2.5.5节和2.5.6节的分析结论可知，以上栈接口的时间复杂度均为常数。

套用以上思路，也可直接基于3.2.2节的List模板类派生出Stack类（习题[4-1]）。

§ 4.2 栈与递归

习题[1-17]指出，递归算法所需的空间量，主要决定于最大递归深度。在达到这一深度的时刻，同时活跃的递归实例达到最多。那么，操作系统具体是如何实现函数（递归）调用的？如何记录调用与被调用函数（递归）实例之间的关系？如何实现函数（递归）调用的返回？又是如何维护同时活跃的所有函数（递归）实例的？所有这些问题的答案，都可归结于栈。

4.2.1 函数调用栈

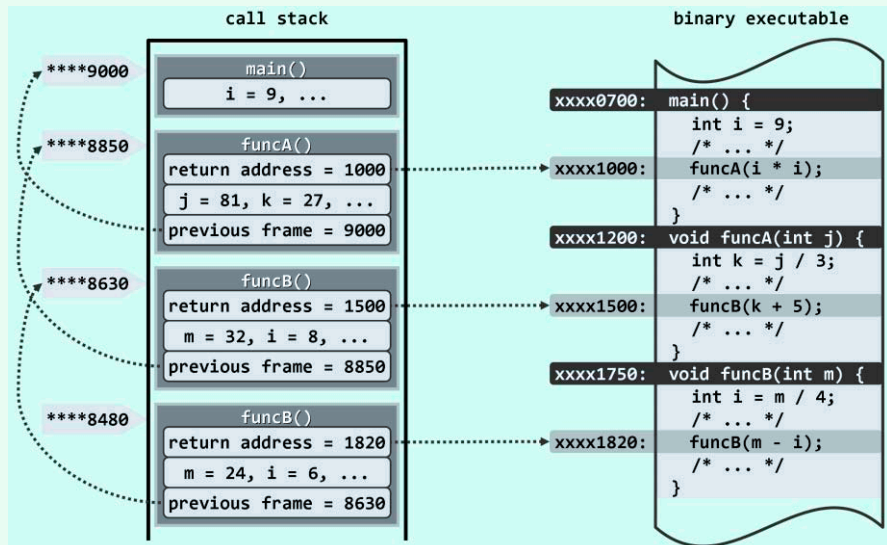


图4.3 函数调用栈实例：主函数main()调用funcA()，funcA()调用funcB()，funcB()再自我调用